Wie doet Wat bij UI/UX Beslissingen

Versie: 1.0
Datum: 2026-03-19
Doelgroep: ITNL02 Team (en andere EAP cohorten)
Context: Sprint 1 loopt vandaag af, dit document geldt voor Sprint 2 en verder
Doel: Verduidelijking wie wat bepaalt bij pagina design en functionaliteit

---

Situatie ITNL02

In Sprint 1 is er veel onduidelijkheid ontstaan over hoe pagina's eruit moeten zien, zowel qua functionaliteit als qua design. Dit document verduidelijkt de rolverdeling tussen Product Owner en andere teamleden.

Vanaf Sprint 2 geldt deze verduidelijking voor alle nieuwe stories en pagina's.

Kernprincipe: WAT vs HOE

Product Owner Bepaalt: WAT en WAAROM

WAT = Functionaliteit, inhoud, gebruikersdoel, business rules
WAAROM = Waarde voor gebruiker, business rationale

Team Bepaalt: HOE

HOE = Design, layout, technische implementatie, user interface details

---

Product Owner Verantwoordelijkheden

1. Functionele Requirements (WAT)

Product Owner bepaalt:

- Welke informatie moet de pagina tonen?
- Welke acties moet een gebruiker kunnen uitvoeren?
- Welke data moet worden ingevoerd/opgeslagen?
- Wat is het doel van de pagina voor de gebruiker?

Voorbeeld - Verlofaanvraag Pagina:

```
✅ PO bepaalt:
- "Pagina moet velden hebben voor: startdatum, einddatum, type verlof, reden"
- "Gebruiker moet aanvraag kunnen versturen"
- "Na versturen ziet gebruiker bevestiging met aanvraag details"
- "Gebruiker moet zijn eerdere aanvragen kunnen zien"
- "Status van aanvraag moet zichtbaar zijn (In behandeling/Goedgekeurd/Afgekeurd)"
```

Wat PO NIET bepaalt:

```
❌ PO bepaalt NIET:
- Of formulier links of rechts staat
- Welke kleur de verstuur-knop heeft
- Exact formaat van datumvelden
- Welke font wordt gebruikt
- Pixel-afmetingen van elementen
```

2. User Experience Requirements (Gebruikersbehoefte)

Product Owner bepaalt:

- Wat moet logisch/intuïtief zijn voor gebruikers?
- Welke informatie is het belangrijkst (prioriteit)?
- Wat zou gebruikers frustreren of verwarren?
- Welke flow moet een gebruiker doorlopen?

Voorbeeld:

```
✅ PO bepaalt:
- "Foutmeldingen moeten duidelijk zijn - gebruiker moet begrijpen wat er fout is"
- "Gebruiker moet direct zien of aanvraag in behandeling is of goedgekeurd"
- "Datum selectie moet simpel zijn - liefst geen handmatig typen van datums"
- "Belangrijkste actie (Indienen) moet prominent aanwezig zijn"
- "Gebruiker moet kunnen annuleren zonder data te verliezen"
```

Wat PO NIET bepaalt:

```
❌ PO bepaalt NIET:
- Exacte positionering van foutmeldingen
- Specifieke kleur voor statusweergave
- Welke datum picker component gebruikt wordt
- Of annuleren-knop links of rechts van indienen-knop staat
```

3. Business Rules & Validaties

Product Owner bepaalt:

- Validatieregels voor data
- Toegangsrechten en autorisatie
- Workflow logica
- GDPR vereisten

Voorbeeld:

```
✅ PO bepaalt:
- "Einddatum moet na startdatum liggen"
- "Gebruiker kan alleen eigen aanvragen zien, niet die van anderen"
- "Aanvraag moet naar manager gestuurd worden voor goedkeuring"
- "Type verlof is verplicht, reden is optioneel"
- "Minimale aanvraag is 1 dag, maximale aanvraag is 30 dagen"
- "Gebruiker kan geen aanvraag indienen voor datums in het verleden"
```

Wat PO NIET bepaalt:

```
❌ PO bepaalt NIET:
- Hoe validatie technisch geïmplementeerd wordt (client-side/server-side)
- Welke error messages framework gebruikt wordt
- Hoe database queries gestructureerd zijn
- Welke API endpoints gebruikt worden
```

Wat Product Owner NIET Bepaalt

Design & Layout (HOE het eruitziet)

Dit is NIET Product Owner verantwoordelijkheid:

- ❌ Exacte kleuren, fonts, font sizes, spacing
- ❌ Waar precies elk element staat (links/rechts/midden/boven/onder)
- ❌ Pixel-perfecte layouts en afmetingen
- ❌ Animaties, transitions, hover states, focus states
- ❌ Specifieke CSS styling
- ❌ Welke UI components/libraries gebruikt worden
- ❌ Responsive breakpoints (wanneer mobile vs desktop layout)
- ❌ Icon keuzes (welk icoon voor welke actie)

Dit is verantwoordelijkheid van: Frontend Developer

Technische Implementatie (HOE het werkt)

Dit is NIET Product Owner verantwoordelijkheid:

- ❌ Welke API endpoints gebruikt worden
- ❌ Hoe data opgeslagen wordt in database (tabellen, velden, relaties)
- ❌ Welke frontend framework/libraries gebruikt worden
- ❌ Code structuur en architectuur
- ❌ Performance optimalisaties
- ❌ Caching strategieën
- ❌ Error handling implementatie
- ❌ Security implementatie details

Dit is verantwoordelijkheid van: Developers (Frontend/Backend/DevOps)

Andere Rollen Bij Pagina Design

Frontend Developer

Verantwoordelijk voor:

- Visual design (kleuren, layout, styling)
- User interface (knoppen, formulieren, navigatie, inputs)
- Component selectie (welke UI library components)
- Responsive design (werkt op mobiel/tablet/desktop)
- Toegankelijkheid (keyboard navigatie, screen readers, ARIA labels)
- Micro-interacties (hover, focus, loading states)

Beslissingen die Frontend Developer maakt:

```
✅ Frontend Developer beslist:
- "Ik plaats het formulier links, aanvraag geschiedenis rechts"
- "Ik gebruik Material UI date picker component voor datum selectie"
- "Ik style knoppen in blauw (#0066CC) met witte tekst"
- "Ik gebruik een tabel voor aanvraag geschiedenis"
- "Ik plaats primary action rechts (Indienen), secondary action links (Annuleren)"
- "Ik gebruik groen voor Goedgekeurd, rood voor Afgekeurd, geel voor In behandeling"
- "Op mobiel toon ik formulier bovenaan, geschiedenis eronder"
```

Frontend Developer vraagt PO feedback op:

- Is deze layout logisch voor gebruikers?
- Zijn belangrijkste elementen zichtbaar/prominent genoeg?
- Is de flow intuïtief?
- Begrijpen gebruikers wat ze moeten doen?

Backend Developer

Verantwoordelijk voor:

- API endpoints voor data ophalen/opslaan
- Business logica validatie (server-side)
- Database schema en queries
- GDPR compliance in data opslag en verwerking
- Security (authentication, authorization)
- Email verzending
- Integration met externe services

Beslissingen die Backend Developer maakt:

```
✅ Backend Developer beslist:
- "POST /api/leave-requests endpoint voor nieuwe aanvraag"
- "GET /api/leave-requests?userId=X voor aanvraag geschiedenis"
- "Ik valideer datums en business rules server-side"
- "Ik sla aanvraag op in 'leave_requests' tabel met foreign key naar 'users'"
- "Ik gebruik TransIP SMTP voor email verzending"
- "Ik log alleen user_id, niet volledige user data in request tabel (privacy by design)"
```

Backend Developer vraagt PO:

- Welke business rules moet ik valideren?
- Welke data moet opgeslagen worden?
- Welke GDPR vereisten gelden?
- Wie heeft toegang tot welke data?

Tester

Verantwoordelijk voor:

- Valideren dat functionaliteit werkt zoals PO specificeerde
- Edge cases identificeren
- Usability feedback geven vanuit gebruikersperspectief
- GDPR compliance valideren

Input die Tester geeft:

```
✅ Tester feedback:
- "Foutmelding 'Invalid input' is onduidelijk - gebruiker begrijpt niet wat fout is"
- "Wat gebeurt er als gebruiker einddatum in het verleden invoert?"
- "Submit knop werkt niet op mobiel (te klein om te klikken)"
- "Gebruiker kan data van andere gebruikers zien in lijst (GDPR issue)"
```

Tester vraagt PO:

- Is dit gedrag correct volgens acceptance criteria?
- Is deze foutmelding acceptabel of moet het duidelijker?
- Welke prioriteit heeft deze bug?

Scrum Master

Verantwoordelijk voor:

- Faciliteren van discussies over rollen en verantwoordelijkheden
- Helpen bij conflicten over wie wat beslist
- Waarborgen dat proces soepel loopt

Scrum Master ondersteunt:

- PO bij verduidelijken van requirements

- Team bij vragen over proces
- Faciliteren van refinement en planning sessies

Hoe Dit in Praktijk Werkt (Vanaf Sprint 2)

Stap 1: Product Owner Schrijft User Story met Acceptance Criteria

Voorbeeld Story:

```
USER STORY: Verlofaanvraag Indienen

Als een medewerker
Wil ik een verlofaanvraag kunnen indienen via een webformulier
Zodat ik verlof kan aanvragen zonder emails te sturen

Acceptance Criteria:
- [ ] Formulier bevat verplichte velden: startdatum, einddatum, type verlof
- [ ] Formulier bevat optioneel veld: reden voor verlof
- [ ] Validatie: einddatum moet minimaal 1 dag na startdatum liggen
- [ ] Validatie: startdatum en einddatum mogen niet in het verleden liggen
- [ ] Validatie: maximale aanvraagperiode is 30 dagen
- [ ] Type verlof opties: Vakantie, Ziekte, Onbetaald, Overig
- [ ] Na succesvol indienen ziet gebruiker bevestigingsscherm met:
  - Aanvraag details (datums, type, reden)
  - Status "In behandeling"
  - Bericht: "Je aanvraag is verstuurd naar je manager"
- [ ] Gebruiker ontvangt email bevestiging met aanvraag details
- [ ] Aanvraag verschijnt in "Mijn Aanvragen" lijst met status "In behandeling"
- [ ] Als validatie faalt, ziet gebruiker duidelijke foutmelding wat er fout is

GDPR Overwegingen:
- Persoonlijke data: gebruiker naam, email, verlof datums, reden
- Juridische basis: Arbeidscontract (noodzakelijk voor uitvoering contract)
- Privacy by design: Alleen medewerker en hun manager kunnen aanvraag zien
- Data retentie: Aanvragen bewaren voor 7 jaar (wettelijke verplichting)

Prioriteit: Hoog
Schatting: [Team schat tijdens refinement]
```

WAT Product Owner WEL specificeert:

- Welke velden nodig zijn
- Welke validaties gelden
- Wat gebruiker ziet na versturen
- Welke data opgeslagen/verstuurd wordt
- GDPR vereisten

WAT Product Owner NIET specificeert:

- Hoe formulier eruit ziet

- Welke kleuren gebruikt worden
- Waar elementen gepositioneerd zijn
- Welke UI components gebruikt worden
- Technische implementatie details

Stap 2: Team Refinement Sessie

Product Owner presenteert story:

> **PO:** "Deze story gaat over het indienen van verlofaanvragen. Belangrijkste is dat medewerkers eenvoudig datums kunnen kiezen, type verlof aangeven, en optioneel een reden toevoegen. Na versturen moeten ze bevestiging zien dat het verstuurd is naar hun manager.
>
> Ik heb acceptance criteria geschreven met alle validaties en vereiste functionaliteit. Jullie kunnen zelf beslissen hoe de pagina eruit ziet - layout, kleuren, components. Maak wat logisch is.
>
> Vragen over de functionaliteit?"

Frontend Developer vraagt:

> **Frontend Dev:** "Hoe moet de pagina layout zijn? Formulier bovenaan, geschiedenis eronder? Of naast elkaar?"

❌ **FOUT Product Owner Antwoord:**

> "Formulier moet links staan, 400px breed, geschiedenis rechts 600px breed, gebruik Material UI components, blauwe primary buttons, 16px Roboto font..."

Waarom fout:

- PO dicteert design details
- PO rolt in Frontend Developer rol
- Team kan geen eigen beslissingen maken
- Verspilt PO tijd aan details

✅ **GOED Product Owner Antwoord:**

> **PO:** "Ik heb geen specifieke voorkeur voor layout. Belangrijkste is dat het duidelijk en simpel is voor gebruikers - ze moeten direct snappen wat ze moeten doen.
>
> Jullie zijn de experts in UI design. Maak een voorstel dat logisch is. Ik geef graag feedback op gebruiksvriendelijkheid - of het intuïtief genoeg is, of belangrijke dingen prominent genoeg zijn, dat soort zaken.
>
> Misschien kunnen jullie een wireframe schetsen voor we gaan bouwen? Dan kan ik feedback geven voordat jullie implementeren."

Waarom goed:

- PO stelt grenzen (duidelijk en simpel)
- PO geeft ownership aan team
- PO biedt aan feedback te geven op UX, niet design details

- PO vraagt om wireframe (goedkope manier om te valideren)

Frontend Developer:

> **Frontend Dev:** "Oké, helder. Ik maak een simpele layout: formulier bovenaan prominent, geschiedenis eronder in collapsible sectie. Gebruik ik Material UI date pickers voor consistentie met rest van applicatie. Ik schets het eerst, laat jullie zien voor ik ga bouwen."

Product Owner:

> **PO:** "Klinkt goed! Laat maar zien zodra je schets hebt."

Backend Developer vraagt:

> **Backend Dev:** "Manager approval is out of scope voor deze story, toch? We slaan alleen op met status 'In behandeling'?"

Product Owner:

> **PO:** "Correct. Manager approval is een aparte story voor later. Voor nu alleen opslaan en email naar manager sturen dat er een aanvraag is. Manager handelt het buiten systeem af."

Tester vraagt:

> **Tester:** "Ik zie validatie voor datums. Moet ik ook testen wat er gebeurt als gebruiker formulier twee keer snel achter elkaar verstuurt? Dubbele aanvragen?"

Product Owner:

> **PO:** "Goed punt! Ja, we moeten voorkomen dat gebruiker per ongeluk dubbele aanvraag maakt. Voeg toe aan acceptance criteria: 'Submit knop disabled tijdens verzenden' en 'Duplicaat detectie: zelfde datums binnen 1 minuut niet toegestaan'."

Tester: "Perfect, dan test ik dat."

Team schat story: 13 punten

Stap 3: Frontend Developer Maakt Wireframe/Schets

Voor implementatie begint, maakt Frontend Developer een simpele wireframe.

Dit kan zijn:

- Hand getekende schets
- Simpel Figma/Excalidraw diagram
- HTML prototype zonder styling
- Beschrijving met ASCII art

Voorbeeld wireframe:

```
┌──────────────────────────────────────────────┐
│  Nieuwe Verlofaanvraag                       |
|  ────────────────────────────────────────   |
|                                              |
|  Startdatum *     [📅 01-04-2026]          |
```

```
│  Einddatum *      [📅 05-04-2026]          │
│                                             │
│  Type verlof *    [Dropdown: Vakantie ▾]   │
│                                             │
│  Reden            [Text area - optioneel]  │
│                   [500 tekens max       ]  │
│                   [                     ]  │
│                                             │
│  [Annuleren]              [Indienen]       │
│                                             │
└─────────────────────────────────────────────┘


┌─────────────────────────────────────────────┐
│  ▼ Mijn Aanvragen (5)                       │
│  ──────────────────────────────────────────  │
│  ┌──────────────────────────────────────┐  │
│  │ 📅 01-04 t/m 05-04 | Vakantie        │ │
│  │ Status: 🟡 In behandeling            │ │
│  │ Ingediend: 19-03-2026                │ │
│  └──────────────────────────────────────┘  │
│  ┌──────────────────────────────────────┐  │
│  │ 📅 15-03 t/m 16-03 | Ziekte          │ │
│  │ Status: ✅ Goedgekeurd               │ │
│  │ Ingediend: 15-03-2026                │ │
│  └──────────────────────────────────────┘  │
└─────────────────────────────────────────────┘


Design keuzes:
- Material UI date picker voor datum selectie
- Dropdown voor type verlof (standaard opties)
- Text area voor reden (optioneel, max 500 chars)
- Primary button rechts (Indienen), secondary links (Annuleren)
- Geschiedenis collapsible (▼) zodat formulier prominent blijft
- Status met emoji's: 🟡 In behandeling, ✅ Goedgekeurd, ❌ Afgekeurd
- Responsive: op mobiel alles onder elkaar, formulier eerst
```

Frontend Developer laat wireframe zien aan team en PO:

In Daily Standup of korte sync:

> **Frontend Dev:** "Ik heb wireframe gemaakt voor verlofaanvraag pagina. Layout logisch? Iets wat ontbreekt uit acceptance criteria perspectief?"

Product Owner bekijkt en geeft feedback:

> **PO:** "Ziet er goed uit! Paar observaties:
>
> ✅ Goed: Formulier is prominent, helder wat verplicht is (*)
> ✅ Goed: Status is duidelijk zichtbaar met emoji's
> ✅ Goed: Geschiedenis is collapsible, houdt focus op nieuw formulier
>
> 💡 Vraag: Status emoji's (🟡 ✅ ❌) zijn visueel duidelijk, maar zijn ze ook toegankelijk voor screen readers? Bijvoorbeeld colorblind gebruikers?

> 💡 Suggestie: In geschiedenis sectie, kan 'Ingediend' datum helpen? Of is dat niet relevant voor gebruiker? Ik twijfel - jullie oordeel."

Frontend Developer:

> **Frontend Dev:** "Goede punten!
>
> - Emoji's: Ik voeg tekst toe naast emoji's, dus: '🟡 In behandeling' niet alleen emoji. Dan werkt het voor screen readers en colorblind gebruikers.
> - Ingediend datum: Handig voor traceren, ik houd het erin. Als we in testing merken dat het niet nuttig is, haal ik het weg.
>
> Verder nog feedback?"

Product Owner:

> **PO:** "Nee, looks good! Go ahead met implementatie."

Waarom dit proces werkt:

- Frontend Dev nam ownership van design
- PO gaf feedback op **gebruiksvriendelijkheid** en **toegankelijkheid**, niet pixels
- Snel (15 minuten discussie), goedkoop (schets vs volledige implementatie)
- Team kan nu bouwen met vertrouwen

Stap 4: Tijdens Ontwikkeling

Frontend Developer bouwt pagina, maakt design keuzes:

- Kiest Material UI date picker component
- Gebruikt team's kleurenschema (blauw #0066CC voor primary buttons)
- Maakt responsive layout (breakpoint op 768px voor mobile)
- Voegt client-side validatie toe (einddatum > startdatum, datums niet in verleden)
- Implementeert disabled state voor submit button tijdens verzenden
- Voegt loading spinner toe tijdens API call

Backend Developer bouwt API:

- `POST /api/leave-requests` endpoint
- Valideert datums en business rules server-side
- Implementeert duplicaat detectie (zelfde datums binnen 1 minuut)
- Stuurt email via TransIP SMTP naar gebruiker en manager
- Slaat op in database met status 'pending'
- Implementeert GDPR logging (wie heeft toegang tot data)

Product Owner rol tijdens ontwikkeling:

✅ **Product Owner DOET:**

- Beschikbaar blijven voor vragen over requirements

- Beantwoorden van clarificatie vragen binnen 24 uur
- Aanwezig zijn bij Daily Standups (of bereikbaar via Teams)

❌ **Product Owner DOET NIET:**

- Meekijken over schouder van developers
- Code reviews doen
- Design details dicteren
- Technische beslissingen maken

Voorbeeld vraag tijdens ontwikkeling:

> **Backend Dev (via Teams):** "PO, als gebruiker dubbele aanvraag probeert (zelfde datums binnen 1 minuut), moet ik oude aanvraag overschrijven of nieuwe weigeren?"

> **PO:** "Nieuwe weigeren. Toon foutmelding: 'Je hebt al een aanvraag voor deze datums ingediend. Wacht 1 minuut en probeer opnieuw.' Dit voorkomt dat gebruiker per ongeluk oude aanvraag overschrijft."

> **Backend Dev:** "Clear, thanks!"

Stap 5: Testing & Feedback

Developer markeert story "Ready for Testing" in Jira.

Tester test eerst (functioneel):

- ✅ Kan datums selecteren → Werkt
- ✅ Kan type verlof kiezen → Werkt
- ✅ Validatie: einddatum voor startdatum → Toont fout ✅
- ✅ Validatie: datum in verleden → Toont fout ✅
- ✅ Bevestigingsscherm toont correcte details → Werkt
- ✅ Email ontvangen → Werkt
- ✅ Aanvraag in geschiedenis lijst → Werkt
- ✅ Duplicaat detectie → Werkt

Tester geeft UX feedback:

> **Tester → PO (via Teams):**
> "Story is functioneel correct, maar één UX issue: Foutmelding bij ongeldige datum is 'Invalid date range'. Voor gebruiker is dit te technisch. Kunnen we dit aanpassen naar 'Einddatum moet na startdatum liggen'?"

> **PO:** "Goed punt! Dit staat in acceptance criteria ('duidelijke foutmelding wat er fout is'). Kan dit aangepast worden?"

> **Frontend Dev:** "Sure, ik pas foutmeldingen aan naar duidelijke Nederlands."

Tester hertest → ✅ Alle acceptance criteria voldaan

Product Owner test vanuit gebruikersperspectief:

PO test happy path:

- ✅ Kan ik verlof aanvragen? → Werkt intuïtief
- ✅ Is het duidelijk wat ik moet doen? → Ja, helder
- ✅ Zie ik bevestiging? → Ja, duidelijk
- ✅ Kan ik mijn aanvragen terugzien? → Ja, overzichtelijk

PO test edge cases:

- ✅ Wat als ik geen reden invul? → Werkt, optioneel veld
- ✅ Wat als ik probeer te versturen met lege velden? → Duidelijke validatie

PO geeft finale UX feedback:

> **PO:** "Één klein puntje: In geschiedenis lijst, status is duidelijk, maar kan manager naam erbij? Zodat gebruiker weet bij wie aanvraag ligt? Of is dat te veel informatie?"

> **Frontend Dev:** "We hebben manager info niet in de huidige API response. Als dit belangrijk is, moet Backend Dev dat toevoegen, wat extra werk is."

> **PO:** "Laat maar voor nu. Nice to have, maar niet kritiek. Story is goed genoeg. Ik accepteer."

Product Owner accepteert story als:

- ✅ Alle acceptance criteria voldaan
- ✅ Definition of Done gehaald (tests, code review, deployed)
- ✅ UX is goed genoeg (niet perfect, maar bruikbaar en duidelijk)
- ✅ GDPR requirements gevalideerd

Story wordt gemarkeerd als Done.

Veel Voorkomende Problemen & Oplossingen (Sprint 2+)

Probleem 1: Product Owner Specificeert Design Details

Symptoom:

Product Owner schrijft in acceptance criteria:

```
❌ FOUT:
- [ ] Submit knop moet 150px breed zijn, blauw (#0066CC), rechtsonder geplaatst
- [ ] Formulier moet 400px breed zijn, gecentreerd op pagina
- [ ] Font moet Roboto 16px zijn
- [ ] Margins tussen velden: 20px
```

Waarom dit fout is:

- PO rolt in Frontend Developer rol
- Team kan niet eigen design beslissingen maken
- Verspilt PO tijd aan details die niet bijdragen aan waarde

- Moeilijk te onderhouden (wat als team ander design system kiest?)
- Beperkt creativiteit en expertise van Frontend Developer

Oplossing:

Product Owner herschrijft acceptance criteria:

```
✅ GOED:
- [ ] Formulier is duidelijk en overzichtelijk - gebruiker begrijpt direct wat te doen
- [ ] Submit actie is prominent aanwezig - gebruiker vindt deze gemakkelijk
- [ ] Visuele hiërarchie: belangrijkste elementen (datums, type) zijn meest opvallend
- [ ] Consistentie met rest van applicatie (zelfde look & feel)
```

Frontend Developer maakt design beslissingen:

- Kiest afmetingen, kleuren, fonts die passen bij design system
- Implementeert responsive layout
- Zorgt voor toegankelijkheid

PO geeft feedback op gebruiksvriendelijkheid, niet pixels.

Probleem 2: Niemand Neemt Design Verantwoordelijkheid

Symptoom:

Team vraagt constant in Daily Standup:

> **Developer:** "Hoe moet deze pagina eruitzien? Waar moet ik de knop plaatsen?"

> **PO:** "Weet ik niet, jullie beslissen."

> **Developer:** "Maar jij bent PO, jij moet het weten!"

> **PO:** "Ik zeg alleen wat het moet doen, niet hoe het eruitziet."

> **Developer:** "Maar dan maak ik iets en jij vindt het niet goed..."

[Deadlock - niemand neemt ownership]

Waarom dit fout is:

- Onduidelijke rol grenzen
- Niemand neemt ownership van design
- Team is bang om foute beslissing te maken
- Verspilt tijd in discussies
- Story's komen niet af

Oplossing:

Scrum Master faciliteert rol verduidelijking:

> **Scrum Master (in Refinement of Standup):**
>
> "Ik zie dat we vastlopen op wie design beslissingen maakt. Laten we dit verduidelijken:

> **Product Owner:** Jij specificeert WAT de pagina moet doen (functionaliteit) en WAAROM (waarde voor gebruiker). Je schrijft acceptance criteria over functionaliteit, niet design.
>
> **Frontend Developer:** Jij maakt design beslissingen - layout, kleuren, componenten. Jij bent de expert.
>
> **Proces vanaf nu:**
>
> 1. PO schrijft functionele acceptance criteria
> 2. Frontend Dev maakt wireframe/schets
> 3. Frontend Dev laat schets zien aan PO (5-10 min review)
> 4. PO geeft feedback op gebruiksvriendelijkheid: Is het logisch? Duidelijk? Intuïtief?
> 5. Frontend Dev implementeert
> 6. PO test en accepteert als functionaliteit werkt en UX goed genoeg is
>
> **Akkoord?"**

Team stemt in.

Vanaf nu in Sprint 2:

- Frontend Dev maakt wireframes VOOR implementatie
- PO bekijkt en geeft feedback binnen 24 uur
- Frontend Dev implementeert met vertrouwen
- Geen endloze discussies over "hoe moet het eruitzien"

Probleem 3: Te Vage User Stories

Symptoom:

```
❌ VAGE STORY:

USER STORY: Gebruiker kan verlof aanvragen

Acceptance Criteria:
- [ ] Gebruiker kan aanvraag indienen
- [ ] Aanvraag wordt opgeslagen

Klaar.
```

Waarom dit fout is:

- Onduidelijk WAT precies nodig is
- Welke velden? Welke validaties? Wat ziet gebruiker?
- Team moet gissen → veel vragen → vertraging
- Veel rework: "Oh ik dacht iets anders"
- Testing is onduidelijk: wat is "correct"?

Oplossing:

Product Owner schrijft specifieke acceptance criteria:

```
✅ SPECIFIEKE STORY:

USER STORY: Medewerker kan verlofaanvraag indienen

Als een medewerker
Wil ik een verlofaanvraag kunnen indienen via een webformulier
Zodat ik verlof kan aanvragen zonder emails te sturen naar HR/manager

Acceptance Criteria - Functionaliteit:
- [ ] Formulier bevat verplichte velden: startdatum, einddatum, type verlof
- [ ] Formulier bevat optioneel veld: reden (max 500 karakters)
- [ ] Type verlof opties: Vakantie, Ziekte, Onbetaald, Verlof zonder behoud van loon, Overig

Acceptance Criteria - Validaties:
- [ ] Einddatum moet minimaal 1 dag na startdatum liggen
- [ ] Startdatum en einddatum mogen niet in het verleden liggen
- [ ] Maximale aanvraagperiode is 30 dagen (31 dagen = fout)
- [ ] Alle verplichte velden moeten ingevuld zijn voor versturen mogelijk is
- [ ] Foutmeldingen zijn duidelijk en in begrijpelijk Nederlands (niet technische codes)

Acceptance Criteria - Na Succesvol Versturen:
- [ ] Gebruiker ziet bevestigingsscherm met volgende info:
  - Ingediende datums
  - Type verlof
  - Reden (als ingevuld)
  - Status: "In behandeling"
  - Bericht: "Je aanvraag is verstuurd naar [Manager Naam]"
- [ ] Gebruiker ontvangt email bevestiging met aanvraag details
- [ ] Manager ontvangt email notificatie over nieuwe aanvraag

Acceptance Criteria - Aanvraag Geschiedenis:
- [ ] Aanvraag verschijnt in "Mijn Aanvragen" sectie
- [ ] Getoond per aanvraag: datums, type, status, ingediend op
- [ ] Status mogelijkheden: In behandeling, Goedgekeurd, Afgekeurd

Acceptance Criteria - Error Handling:
- [ ] Als API call faalt, ziet gebruiker: "Er ging iets fout. Probeer later opnieuw."
- [ ] Als duplicaat detectie triggert: "Je hebt al een aanvraag voor deze datums. Wacht 1
minuut."
- [ ] Gebruiker verliest GEEN data bij foutmelding (velden blijven gevuld)

GDPR Overwegingen:
- Persoonlijke data: naam, email, verlof datums, reden voor verlof
- Juridische basis: Arbeidscontract (noodzakelijk voor uitvoering arbeidsovereenkomst)
- Privacy by design:
  - Alleen medewerker zelf en hun direct manager kunnen aanvraag zien
  - Reden voor verlof is optioneel (gebruiker bepaalt of ze dit delen)
  - Aanvraag data wordt niet gedeeld met andere afdelingen zonder toestemming
- Data retentie: Verlof aanvragen bewaren voor 7 jaar (fiscale bewaarplicht)

Dependencies:
```

```
- Vereist: User authentication (gebruiker moet ingelogd zijn)
- Vereist: Manager relatie in database (wie is manager van gebruiker?)
- Vereist: TransIP SMTP configuratie voor email verzending

Out of Scope (voor latere stories):
- Manager kan aanvraag goedkeuren/afkeuren (aparte story)
- Notificaties bij status wijziging (aparte story)
- Aanvraag bewerken/annuleren (aparte story)
- Verlof tegoed berekening (aparte story)

Prioriteit: Hoog (kernfunctionaliteit applicatie)
Schatting: [Team schat tijdens refinement]
```

WAT dit oplost:

- Team weet exact wat te bouwen
- Minder vragen tijdens sprint
- Tester weet precies wat te valideren
- Minder rework
- Duidelijke scope (wat WEL en NIET in deze story)

WAT PO nog steeds NIET specificeert:

- Hoe formulier eruit ziet
- Kleuren, fonts, spacing
- Welke components gebruikt worden
- Technische implementatie

Team heeft nog steeds volledige ownership over HOE.

Probleem 4: Product Owner Geeft Tegenstrijdige Feedback

Symptoom:

In Refinement (week 1):

> **PO:** "Maak het simpel en minimalistisch. Geen onnodige velden."

In Sprint Review (week 2):

> **PO:** "Waarom staat er geen veld voor 'aanvraag prioriteit'? Dat moet erbij!"

Developer:

> "Maar je zei simpel en minimalistisch? En het stond niet in acceptance criteria..."

PO:

> "Maar het is toch logisch dat prioriteit erbij moet?"

Team:

> *[Frustratie - wat wil PO nou eigenlijk?]*

Waarom dit fout is:

- PO wijzigt requirements NADAT team gebouwd heeft
- Team gefrustreerd: "We kunnen nooit goed doen"
- Verspilt tijd aan rework
- Velocity onvoorspelbaar

Oplossing:

Product Owner praktijken:

1. Schrijf volledige acceptance criteria VOOR sprint:

Als PO iets belangrijk vindt, moet het in acceptance criteria staan VOOR Sprint Planning.

2. Onderscheid "must have" vs "nice to have":

```
Acceptance Criteria (Must Have):
- [ ] Veld X, Y, Z aanwezig
- [ ] Validatie A, B, C werkt

Nice to Have (niet in deze sprint):
- Veld voor prioriteit (toevoegen in volgende sprint als waardevol)
- Automatische manager suggestie
```

3. Wijzigingen = nieuwe story:

Als PO tijdens Sprint Review iets mist dat niet in acceptance criteria stond:

> **PO:** "Ik zie dat er geen prioriteit veld is. Dit is waardevol - ik maak een nieuwe story voor volgende sprint om dit toe te voegen. Deze story voldoet aan de acceptance criteria die we afspraken, dus ik accepteer hem."

NIET:

> "Dit moet erbij, pas het aan!" (scope creep)

4. Verantwoordelijkheid nemen:

Als PO iets vergat te specificeren:

> **PO:** "Sorry, ik had prioriteit veld moeten toevoegen aan acceptance criteria. Mijn fout. Dit neem ik mee voor volgende story."

NIET:

> "Jullie hadden moeten vragen of dit erbij moest!" (blame shifting)

Acties voor ITNL02 Team (Sprint 2 en verder)

Voor Product Owner

Onmiddellijk (Voor Sprint 2 Planning):

1. Verduidelijk rollen met team (15 minuten in Daily Standup of kort overleg):

> **PO aan team:**
>
> "Ik wil iets verduidelijken over mijn rol, relevant voor Sprint 2 en verder.
>
> **Mijn rol:**
>
> - Ik specificeer **WAT** pagina's moeten doen (functionaliteit, data, business rules, GDPR)
> - Ik specificeer **WAAROM** (waarde voor gebruiker, business rationale)
> - Ik schrijf acceptance criteria die functionaliteit beschrijven
>
> **Mijn rol is NIET:**
>
> - Bepalen **HOE** dingen eruitzien (kleuren, layout, fonts, componenten)
> - Technische implementatie beslissingen (API endpoints, database schema, code)
>
> **Jullie rol (Frontend/Backend/Tester):**
>
> - Frontend: Jullie maken design beslissingen - layout, styling, components
> - Backend: Jullie maken technische beslissingen - API design, database, architectuur
> - Tester: Jullie valideren functionaliteit en geven UX feedback
>
> **Nieuw proces vanaf Sprint 2:**
>
> 1. Ik schrijf functionele acceptance criteria (WAT moet werken)
> 2. Jullie maken wireframe/schets voor complexe pagina's (optioneel maar geadviseerd)
> 3. Jullie laten me schets zien (5-10 min review)
> 4. Ik geef feedback op gebruiksvriendelijkheid: Is het logisch? Duidelijk? Intuïtief?
> 5. Jullie implementeren met vertrouwen
> 6. Ik test en accepteer als functionaliteit correct is en UX goed genoeg
>
> **Verwachtingen:**
>
> - Van mij: Heldere acceptance criteria, beschikbaar voor vragen, snelle feedback op wireframes
> - Van jullie: Neem ownership van design/techniek, vraag als iets functioneel onduidelijk is
>
> Vragen? Onduidelijkheden?"

2. Review alle Sprint 2 stories:

Ga door alle stories voor Sprint 2 en check:

- [ ] Zijn acceptance criteria functioneel, niet design-specifiek?
- [ ] Is duidelijk WAT moet werken, niet HOE het eruitziet?
- [ ] Zijn validaties en business rules gespecificeerd?

- [ ] Zijn GDPR overwegingen gedocumenteerd?
- [ ] Is out of scope duidelijk (wat NIET in deze story)?

Herschrijf waar nodig:

```
❌ Voor Sprint 2, verwijder:
"Knop moet rechts staan"
"Gebruik blauw kleurenschema"
"Tabel moet 600px breed zijn"

✅ Voor Sprint 2, behoud/voeg toe:
"Gebruiker moet actie kunnen annuleren"
"Foutmeldingen in duidelijk Nederlands"
"Status moet zichtbaar zijn"
"Validatie: veld X moet formaat Y hebben"
```

3. Communiceer verwachtingen over wireframes:

> **PO aan Frontend Developer(s):**
>
> "Voor complexe pagina's (nieuw formulier, dashboard, etc.), kan je een wireframe maken voor je gaat bouwen? Hoeft niet fancy - simpele schets, Excalidraw, of beschrijving is genoeg.
>
> Ik review binnen 24 uur en geef feedback op:
>
> - Is layout logisch voor gebruikers?
> - Zijn belangrijke dingen prominent genoeg?
> - Missen we iets functioneels?
>
> Dit bespaart ons allebei tijd - beter om op schets fase te valideren dan na volledige implementatie.
>
> Akkoord?"

Voor Frontend Developer(s)

Vanaf Sprint 2:

1. Neem ownership van design beslissingen:

- Je bepaalt: layout, kleuren, fonts, components, responsive breakpoints
- Je vraagt PO NIET: "Welke kleur moet knop zijn?" of "Waar moet dit staan?"
- Je vraagt PO WEL: "Is deze layout logisch voor gebruikers?" (na wireframe)

2. Maak wireframes voor complexe pagina's:

Voor nieuwe pagina's of complexe features:

- Maak simpele wireframe VOOR implementatie
- Laat aan team en PO zien (in Refinement of korte sync)
- Vraag: "Layout logisch? Iets belangrijks dat ontbreekt?"
- Implementeer na feedback

3. Stel functionele vragen, niet design vragen:

```
✅ Goede vragen aan PO:
"Moet gebruiker kunnen annuleren zonder data te verliezen?"
"Wat ziet gebruiker als API call faalt?"
"Welke velden zijn verplicht?"
"In welke situaties moet validatie triggeren?"

❌ Vermijd vragen aan PO:
"Welke kleur voor submit button?"
"Moet formulier links of rechts?"
"Welke font size?"
"Hoeveel padding tussen elementen?"
```

4. Implementeer met design system consistentie:

- Gebruik zelfde components als rest van applicatie
- Volg bestaande kleurenschema en styling patterns
- Zorg voor toegankelijkheid (ARIA labels, keyboard navigatie)
- Test responsive design

Voor Backend Developer(s)

Vanaf Sprint 2:

1. Vraag PO naar business logic, niet implementatie:

```
✅ Goede vragen aan PO:
"Welke validaties moeten server-side gecheckt worden?"
"Wat gebeurt er als gebruiker geen toegang heeft tot deze data?"
"Welke GDPR regels gelden voor deze feature?"
"Wie mag deze data zien/wijzigen?"

❌ Vermijd vragen aan PO:
"Welke database tabel moeten we gebruiken?"
"REST of GraphQL?"
"Welke HTTP status codes?"
```

2. Neem ownership van technische beslissingen:

- API design (endpoints, request/response formats)
- Database schema
- Validatie implementatie
- Security maatregelen
- Performance optimalisaties

3. Communiceer GDPR implementatie naar PO:

> **Backend Dev naar PO:**
> "Voor verlofaanvraag feature implementeer ik:
>
> - Alleen user_id opslaan in request tabel, niet volledige user object (minimale data)
> - Access control: gebruiker kan alleen eigen requests zien
> - Audit log: bijhouden wie wanneer toegang had tot data
> - Retention: automatische cleanup na 7 jaar
>
> Voldoet dit aan GDPR requirements?"

> **PO:** "Klinkt goed! Check met staff/DPO voor zekerheid."

Voor Tester

Vanaf Sprint 2:

1. Test functionaliteit tegen acceptance criteria:

- Valideer dat elk acceptance criterium werkt zoals gespecificeerd
- Test edge cases en error scenarios
- Verifieer GDPR compliance

2. Geef UX feedback aan PO:

Als je tijdens testen UX issues ziet:

> **Tester naar PO:**
> "Functionaliteit werkt, maar foutmelding is onduidelijk. Gebruiker begrijpt niet wat fout is. Suggestie: verander 'Error 400' naar 'Einddatum moet na startdatum liggen'."

> **PO:** "Goed punt, dit valt onder acceptance criterium 'duidelijke foutmeldingen'. Kan dit aangepast worden?"

3. Valideer toegankelijkheid:

Test of pagina:

- Werkt met keyboard navigatie (tab, enter, esc)
- Screen reader compatible is (ARIA labels)
- Kleurcontrast voldoende is (colorblind toegankelijk)

Voor Scrum Master

Vanaf Sprint 2:

1. Faciliteer rol verduidelijking als nodig:

Als je ziet dat team/PO discussiëren over "wie beslist wat":

- Verwijs naar dit document
- Faciliteer kort gesprek (10-15 min) om grenzen te verduidelijken

- Help team tot afspraken komen

2. Bewaakt proces:

- Zorgt dat PO acceptance criteria schrijft VOOR Sprint Planning
- Zorgt dat team tijd krijgt voor wireframes (if needed)
- Zorgt dat feedback cycli snel gaan (PO antwoord binnen 24u)

3. Signaleer anti-patronen:

Als je ziet dat:

- PO design details specificeert → "PO, focus op WAT, niet HOE"
- Team niet durft design beslissingen te maken → "Team, jullie zijn experts, maak voorstel"
- Veel rework door onduidelijke requirements → "PO, kunnen acceptance criteria specifieker?"

Sprint 2 Planning Checklist

Voor Product Owner (Voor Planning):

- [ ] Alle Sprint 2 stories hebben functionele acceptance criteria (WAT, niet HOE)
- [ ] GDPR overwegingen gedocumenteerd waar relevant
- [ ] Out of scope expliciet vermeld per story
- [ ] Business rules en validaties gespecificeerd
- [ ] Rollen en verwachtingen gecommuniceerd aan team
- [ ] Commitment: beschikbaar voor vragen, wireframe feedback binnen 24u

Voor Frontend Developer (In Planning):

- [ ] Begrijp dat je ownership hebt over design beslissingen
- [ ] Plan tijd voor wireframes bij complexe pagina's
- [ ] Commitment: laat wireframes zien voor implementatie (bij complexe features)
- [ ] Vraag PO verduidelijking over functionele requirements (niet design)

Voor Backend Developer (In Planning):

- [ ] Begrijp business rules en validaties uit acceptance criteria
- [ ] Identificeer GDPR vereisten uit stories
- [ ] Plan API design en database changes
- [ ] Vraag PO verduidelijking over business logic (niet technische implementatie)

Voor Tester (In Planning):

- [ ] Begrijp acceptance criteria - wat moet getest worden
- [ ] Identificeer edge cases die niet expliciet vermeld zijn
- [ ] Plan test scenarios (happy path, sad path, edge cases)
- [ ] Commitment: geef UX feedback tijdens testing

Voor Scrum Master (In Planning):

- [ ] Faciliteer rol verduidelijking als nodig
- [ ] Zorg dat team begrijpt WAT vs HOE scheiding
- [ ] Check of acceptance criteria specifiek genoeg zijn
- [ ] Zorg dat team commitment maakt over wireframes/feedback proces

---

Voorbeeld Dialogen (Sprint 2 en verder)

Voorbeeld 1: Functionele Vraag (GOED)

Context: Developer werkt aan verlofaanvraag pagina

> **Developer (Teams):** "PO, als gebruiker een aanvraag indient en de API geeft error 500 (server probleem), wat moet gebruiker zien? Alleen 'Er ging iets fout' of meer details?"

> **PO:** "Goede vraag. Toon: 'Er ging iets fout bij het versturen van je aanvraag. Probeer het over een paar minuten opnieuw. Als het probleem blijft, neem contact op met IT support.'
>
> Belangrijkste: gebruiker begrijpt dat het niet hun fout is, en weet wat te doen. Specifieke technische error details (500, API endpoint, etc.) niet tonen aan gebruiker."

> **Developer:** "Clear, thanks!"

Waarom goed:

- Developer vraagt naar GEDRAG (wat moet gebruiker zien)
- PO specificeert FUNCTIONALITEIT (welke informatie, welke actie)
- PO specificeert GEEN design (hoe foutmelding eruit ziet)

Voorbeeld 2: Design Vraag (FOUT → GECORRIGEERD)

Context: Frontend Developer werkt aan dashboard

> **Frontend Dev:** "PO, moet het dashboard 2 kolommen of 3 kolommen hebben?"

❌ **FOUT PO Antwoord:**

> "3 kolommen, elk 33% breed, eerste kolom voor statistieken, tweede voor recente aanvragen, derde voor meldingen."

Waarom fout: PO dicteert layout details

✅ **GOED PO Antwoord:**

> "Ik heb geen specifieke voorkeur voor aantal kolommen. Belangrijkste is dat gebruiker snel kan zien:
>
> 1. Hun verlof statistieken (hoeveel dagen gebruikt/over)
> 2. Status van recente aanvragen
> 3. Belangrijke meldingen
>
> Hoe je dat layout maakt is jouw expertise. Maak wat logisch is, laat me zien, dan geef ik feedback op of het duidelijk genoeg is."

> **Frontend Dev:** "Oké, ik maak een 2-kolom layout (statistieken links, aanvragen/meldingen rechts gestacked). Op mobiel alles onder elkaar. Ik laat schets zien voor ik implementeer."

> **PO:** "Klinkt goed!"

Waarom goed:

- PO specificeert WAT belangrijk is (welke informatie)
- PO geeft ownership aan Frontend Dev voor HOE (layout)
- Frontend Dev maakt voorstel
- PO biedt feedback aan

Voorbeeld 3: Wireframe Review (GOED)

Context: Frontend Developer toont wireframe voor aanvraag detail pagina

> **Frontend Dev:** "Ik heb wireframe voor aanvraag detail pagina. Feedback?"

```
Wireframe:
┌──────────────────────────────────┐
| Verlofaanvraag Details           |
|                                  |
| Status: 🟡 In behandeling        |
|                                  |
| Datums: 01-04 t/m 05-04         |
| Type: Vakantie                   |
| Reden: Familie bezoek            |
|                                  |
| Ingediend op: 19-03-2026        |
| Ingediend bij: Jan Bakker       |
|                                  |
| [Annuleer Aanvraag]             |
└──────────────────────────────────┘
```

PO Review:

> **PO:** "Layout is logisch en helder. Paar opmerkingen:
>
> ✅ Goed: Status prominent bovenaan - gebruiker ziet meteen waar aanvraag staat
> ✅ Goed: Alle relevante info aanwezig

> 💡 Vraag: 'Annuleer Aanvraag' - kunnen we deze alleen tonen als status 'In behandeling' is? Als al goedgekeurd/afgekeurd, heeft annuleren geen zin. Of zie ik dat verkeerd?
>
> 💡 Vraag: Moet er geschiedenis zijn van status wijzigingen? Bijv. 'Goedgekeurd op 20-03 door Jan Bakker'? Of is huidige status voldoende?"

> **Frontend Dev:**
> "Goede punten!
>
> - Annuleer knop: Ja, logisch om alleen bij 'In behandeling' te tonen. Ik voeg die conditie toe.
> - Geschiedenis: Dat staat niet in acceptance criteria voor deze story. Is dat nice-to-have of must-have?"

> **PO:** "Nice-to-have. Niet voor deze sprint. Ik maak aparte story 'Aanvraag geschiedenis tonen' voor later. Voor nu is huidige status voldoende."

> **Frontend Dev:** "Oké, dan implementeer ik zoals wireframe, met annuleer knop alleen bij 'In behandeling' status."

> **PO:** "Perfect, go ahead!"

Waarom goed:

- Frontend Dev liet wireframe zien VOOR implementatie
- PO gaf feedback op FUNCTIONALITEIT en LOGICA, niet pixels
- PO onderscheidde must-have vs nice-to-have
- Snelle cyclus (10 min discussie) voorkomt rework later

Voorbeeld 4: Acceptance Criteria Onduidelijk (GECORRIGEERD)

Context: Developer leest story in Sprint Planning

> **Developer:** "Story zegt 'Gebruiker moet aanvraag kunnen annuleren'. Maar wanneer kan dat? Altijd? Ook als al goedgekeurd?"

> **PO:** "Ah, goed punt! Ik was niet specifiek genoeg. Laat me acceptance criteria aanvullen:
>
> **UPDATED:**
>
> - [ ] Gebruiker kan aanvraag annuleren als status 'In behandeling' is
> - [ ] Na annuleren: status wordt 'Geannuleerd'
> - [ ] Na annuleren: manager krijgt email notificatie
> - [ ] Gebruiker kan NIET annuleren als status 'Goedgekeurd' of 'Afgekeurd' is
> - [ ] Als gebruiker probeert te annuleren bij verkeerde status: toon melding 'Je kunt alleen aanvragen in behandeling annuleren'
>
> Duidelijker?"

> **Developer:** "Ja, helder nu. Thanks!"

Waarom goed:

- Developer signaleerde onduidelijkheid VOOR bouwen

- PO nam verantwoordelijkheid en verduidelijkte direct
- Acceptance criteria aangepast VOOR implementatie (geen rework)

Samenvatting: WAT vs HOE

✅ Product Owner Bepaalt (WAT & WAAROM):

Functionaliteit:

- Welke velden, knoppen, acties beschikbaar zijn
- Welke data getoond/opgeslagen wordt
- Welke gebruikersrollen welke acties kunnen doen

Business Rules:

- Validaties (welke input is geldig/ongeldig)
- Workflows (wat gebeurt na actie X)
- Toegangsrechten (wie mag wat zien/doen)

User Experience:

- Wat intuïtief/logisch moet zijn
- Welke informatie het belangrijkst is
- Wat gebruikers frustreert (wat vermijden)

GDPR:

- Welke data persoonlijk is
- Privacy by design vereisten
- Toegangscontroles
- Data retentie regels

✅ Frontend Developer Bepaalt (HOE - Design):

Visual Design:

- Kleuren, fonts, font sizes, spacing
- Layout (waar elementen staan)
- Responsive design (mobile/tablet/desktop)
- Iconography

UI Components:

- Welke components gebruikt worden (buttons, inputs, dropdowns, etc.)
- Welke UI library (Material UI, Bootstrap, custom)
- Animaties en transitions
- Hover/focus states

Toegankelijkheid:

- ARIA labels
- Keyboard navigatie
- Kleurcontrast
- Screen reader compatibility

✅ Backend Developer Bepaalt (HOE - Techniek):

API Design:

- Endpoints (REST, GraphQL, etc.)
- Request/response formats
- HTTP methods en status codes
- Error response formats

Database:

- Schema (tabellen, velden, relaties)
- Indexes en optimalisaties
- Queries

Implementatie:

- Validatie implementatie (server-side)
- Security maatregelen
- Caching strategieën
- Performance optimalisaties

✅ Tester Bepaalt (HOE - Kwaliteit):

Test Strategy:

- Welke test scenarios (happy path, edge cases, errors)
- Test data
- Test volgorde

Quality Feedback:

- UX issues (onduidelijke foutmeldingen, verwarrende flows)
- Edge cases die niet in acceptance criteria staan
- Toegankelijkheid problemen
- GDPR compliance validatie

Sprint 2 Succesindicatoren

Je weet dat rolverdeling werkt als:

✅ **Minder vragen over "hoe moet het eruitzien":**

- Team maakt design beslissingen zelfstandig
- PO wordt niet gevraagd over pixels/kleuren

✅ **Snellere story completion:**

- Minder rework door onduidelijke requirements
- Team bouwt met vertrouwen

✅ **Minder frustratie:**

- Team voelt ownership over hun expertise gebied
- PO kan focussen op waarde/prioriteit, niet details

✅ **Betere kwaliteit:**

- Acceptance criteria zijn helder VOOR sprint
- Wireframes valideren UX VOOR implementatie
- Minder bugs door duidelijke requirements

✅ **Positieve retrospective feedback:**

- "Requirements waren duidelijker deze sprint"
- "Ik kon design beslissingen maken zonder constant te vragen"
- "PO was beschikbaar voor functionele vragen, maar liet ons HOE bepalen"

Vragen & Contact

Voor verduidelijking over dit document:

- Vraag je Scrum Master
- Bespreek in Daily Standup of Refinement
- Neem contact op met staff

Voor verduidelijking tijdens Sprint 2:

- Functionele vragen → Vraag Product Owner
- Design vragen → Bespreek met Frontend Developer / Team
- Technische vragen → Bespreek met Backend Developer / Team
- Proces vragen → Vraag Scrum Master

Document Einde

Versie: 1.0
Geldig vanaf: Sprint 2 (Sprint 1 loopt vandaag af)
Review datum: Na Sprint 2 Retrospective (aanpassen indien nodig)